Applying Established Software Engineering Practices to Agentic AI Development: Autonomy, Guardrails and Human Oversight

Samuel J. Harries

Student ID: 730039610

Abstract

Agentic artificial intelligence (AI) systems are increasingly capable of performing substantial software development tasks, including code generation, testing, and debugging. However, as these systems undertake larger portions of software implementation, questions arise regarding how established software engineering practices should be maintained, which responsibilities should remain human led, and how agent behaviour should be managed. This report reviews existing research into established software engineering principles, agentic development workflows, human oversight, and technical guardrails, examining their approaches, limitations and methods of evaluation. The literature identifies a need for further empirical investigation into the combined effectiveness of these practices in agent-assisted development. To address this, the project proposes a structured development workflow incorporating human approval stages, technical restrictions, and software verification procedures. The workflow will be applied to the development of a volunteer accommodation booking system for Llechfan. Finally, the report presents a project specification outlining the proposed methodology, deliverables, timeline, and evaluation criteria for investigating the effectiveness and trade-offs of structured agentic development.

Figures and other items such as Gantt chart, critical path diagram etc still need to be created. The figures currently used are not final.

Signature: _____

1 Introduction

With the rise of large language models (LLMs), more and more software engineers are utilising their capabilities in almost all aspects of their work, including code generation, debugging, and testing [1]. Agentic AI systems have extended the capabilities by allowing agents to interact with development tools, modify code, and execute tests with configurable degrees of autonomy [2].

Software engineering involves more than simply producing functional code. Established practices cover requirements engineering, system design, implementation, testing, and maintenance [3]. Whilst benchmarks such as SWE-bench assess the agent's ability to solve practical programming issues, these benchmarks do not consider whether the software engineering practices have been followed adequately [4].

Questions arise about how much autonomy the agents should be given as they become more capable of undertaking more development tasks, and when human intervention is necessary. Research into human-in-the-loop development [5] and runtime guardrails [6] explore different approaches at attempting to control agent behaviour. Whilst these approaches yield success, they produce potential trade-offs between agent autonomy, reliability, human effort, and human intervention frequency.

This literature review aims to examine how established software engineering practices can be applied to agentic AI development, focusing on autonomy, guardrails, and human oversight. Through the use of critical comparison of the existing literature, it aims to contribute to the creation of a structured agent-assisted development workflow, which will be applied to the development of a volunteer accommodation booking system.

2 Established Software Engineering Practices

The Guide to the Software Engineering Body of Knowledge (SWEBOK), published by the IEEE Computer Society, provides a reference framework for established software engineering practices. Its fourth edition shows 18 knowledge areas covering tasks such as requirements engineering, software architecture, design, construction, testing, and maintenance [3]. These areas demonstrate that software engineering is more than implementation, it's an amalgamation of processes that are necessary to develop and maintain software systems.

A fundamental aspect of established software engineering is ensuring that software meets the requirements and maintains an appropriate level of production quality. SWEBOK identifies practices relating to requirements validation, software testing, quality assurance and configuration management. These activities help developers identify defects, manage changes, and evaluate a system to ensure it meets the intended functionality [3]. These practices remain relevant regardless of whether implementation is performed by a human developer or an AI agent.

Although SWEBOK provides a useful foundation for understanding established software engineering, it does not establish how these practices should be applied to autonomous AI development. As it is not an empirical evaluation, it cannot determine which development responsibilities can safely be delegated to AI agents. Even with this limitation, it can be used to provide a point of reference against which agentic development workflows can be examined, especially when investigating requirements, testing, software quality, and human responsibility.

3 Agentic AI in Software Engineering

Large language models have been used in many software engineering activities. Hou et al. [1] conducted a systematic literature review of 395 studies, reviewing applications of LLMs across requirements engineering, software development, testing, and maintenance. However, their review only considers LLM-based approaches broadly, and not specifically on autonomous development agents. Agentic systems extend these applications by allowing these models to interact with development environments and allowing them to undertake multi-step tasks with less developer involvement.

Jimenez et al. [4] introduced SWE-bench, a benchmark evaluating models against practical, real-world GitHub issues. Building from this approach, Yang et al. [2] developed SWE-agent, which makes

use of a specialised Agent-Computer Interface to navigate repositories, modify files, and execute tests. Their findings demonstrated how the tools available to agents can influence their performance. However, both studies primarily focus on resolving individual programming issues rather than managing the wider software engineering lifecycle.

This limitation is explored more specifically by Zeng et al. [7], who evaluated agents undertaking end-to-end software development tasks. Their findings show that agents continue to struggle with requirements fulfilment and self-verification. In conjunction with the issue resolution tasks reviewed by SWE-bench, this provides a broader perspective on agent capabilities. Collectively, these studies suggest that successful implementation does not necessarily demonstrate that an agent can independently fulfil all aspects of a software development project.

4 Structured Development Workflows and Agent Autonomy

By using structured development workflows, autonomous agents and their responsibilities can be organised and managed effectively. Hong et al. [8] introduced MetaGPT, a multi-agent framework that uses Standard Operating Procedures (SOPs) to organise software development tasks. By assigning specialised roles, including architects and engineers, the framework aims to improve coordination and reduce errors between the agents. The study shows improvements in the coherence of the generated solutions, but the results cannot show that the generated software will meet all established software engineering standards.

Whilst MetaGPT focuses on coordination between the agents, Yang et al. [2] investigate how the interfaces available to an agent influence its performance. SWE-agent uses a specialised Agent-Computer Interface to support repository navigation, code modification, and testing. The findings they show suggest that interface design can influence an agent's ability to complete development tasks. Although, providing effective development tools does not establish appropriate boundaries for the desired autonomous behaviour.

A separate approach is presented by Takerngsaksiri et al. [5], whose HULA framework incorporates human oversight into agent-assisted development. Opposed to MetaGPT's primarily agent-driven collaboration methods, HULA allows developers to review and approve coding plans and generated changes before proceeding. Its evaluation suggests benefits for straightforward development tasks, but concerns with code quality remain prevalent. Using these two studies, it demonstrates two distinct approaches to structuring agentic development, but fail to establish what balance of autonomy and human involvement is most effective. This distinction is relevant when designing a workflow that aims to maintain established software engineering practices.

5 Human Oversight, Guardrails and Code Review

Whilst human oversight allows for developers to intervene in agentic development, the review checkpoints do not guarantee software quality. In the industrial evaluation of HULA, Takerngsaksiri et al.[5] reported that 82% of successfully generated plans were approved by the developers. Even though there is a high acceptance rate, this does not establish that the implemented plans were correct or maintainable. The study also shows continuing concerns around the code quality, and is suggested that the effectiveness of human oversight should reviewed through the changes and corrections made by the developers, rather than solely through the approval rates.

Technical guardrails provide another means of controlling agent behaviour without requiring human intervention for every action it takes. Wang et al. [6] introduced AgentSpec, which allows the developers to define rules that will be enforced during agent execution. Opposed to prompt-based instructions, runtime enforcement can actively restrict actions that could violate predefined conditions. Their evaluation reported preventing unsafe executions in over 90% of the chosen code-agent cases. However, the effectiveness of the restrictions set depended on the rules being defined appropriately. Moreover, preventing prohibited actions does not show that an agent's implementation meets the functional requirements, meaning runtime enforcement cannot replace software testing or review of the software.

The effectiveness of autonomous code review was investigated by Cihan et al. [9], who examined 4,335 pull requests, where 1,568 of which received automated reviews. The study shows that 73.8% of automated review comments were marked as resolved, but the average time for a pull request to be closed increased from 5 hours 52 minutes to 8 hours 20 minutes. As shown in Figure 1, the changes in closure time varied between projects, with one project experiencing a reduction. This suggests that there are potential trade-offs between additional review and development efficiency, but the observational results do not establish that automated review alone caused the increase. Additionally, resolving comments does not necessarily show improved code correctness. The need to combine different forms of supervision is further explored by Stolze and Strässle [10], who identified three layers of control, preventive guardrails, executable checks, and human oversight. Their findings suggest that relying on code review alone may become insufficient as AI-generated development increases. However, the study is based on interviews with five practitioners rather than a controlled experiment, limiting the conclusions that can be formed about the effectiveness of this approach. Whilst layered supervision may address different development risks, further evaluation is needed to establish its impact on software quality and human effort.

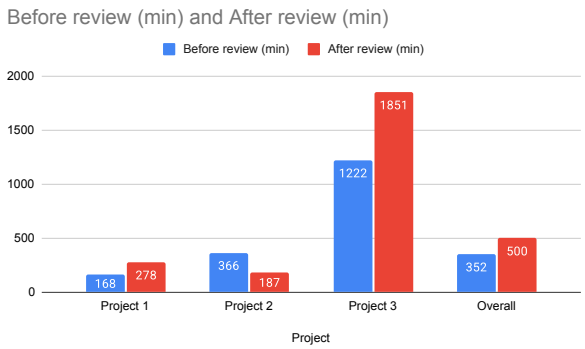


Figure 1: Average pull request closure times before and after automated code review, adapted from Cihan et al. [9].

6 Agentic AI Across the Software Development Lifecycle

The applications of LLMs vary across the software development lifecycle. Hou et al. [1] found that software development accounted for 56.65% of the applications reviewed in their systematic review, compared to 3.9% for requirements engineering and only 0.92% for software design. This implies that research has focused primarily on implementation rather than earlier development activities. Conversely, as the review considers LLM applications broadly, these findings cannot automatically be generalised to autonomous development agents

Requirements engineering is particularly important because errors at this stage can affect subsequent development. Pasquale et al. [11] investigated the use of three LLMs to generate requirements specifications and user stories from previously collected information. Their findings suggest that LLMs can reduce the effort required to produce documentation, however the quality of their outputs depends heavily on the information provided. The literature also shows missing requirements and varying levels of detail between generated user stories. The research focuses on producing specifications from existing documents, rather than identifying stakeholders independent needs. This implies that human intervention remains necessary to validate requirements before implementation begins.

Similarly, concerns around system design arise. Zeng et al. [7] compared different agent architectures for end-to-end software development and found that introducing a dedicated design agent did not improve performance. Their research suggests that incorrect design plans could be treated as authoritative by future coding agents, causing errors to propagate throughout the implementation process. Although these findings are specific to the studied agent configurations, they challenge the assumption that dividing development into more specialised agent roles will always produce better

results.

During implementation and testing, maintaining a connection to the source requirements remains important. Whilst HULA [5] begins with predefined development issues and incorporates feedback during coding, Zeng et al. [7] identifies omitted requirements and ineffective self-verification as major problems in end-to-end development. Collectively, these studies highlight the importance of checking generated implementations against the original requirements, rather than relying on the agent's own assessments. For the proposed workflow, this shows that requirements, implementation, and testing should remain connected through clearly defined acceptance criteria and verification procedures.

7 Evaluating Agentic Software Engineering

Evaluating agentic software development requires measures that can reliably determine whether the generated software is successful. Jimenez et al. [4] use executable tests within SWE-bench to assess whether an agent has resolved a given programming issue. This provides a reproducible and measurable approach to evaluating functional correctness. However, the benchmark primarily measures issue resolution, without assessing aspects such as maintainability, development processes, or adherence to established software engineering practices.

A broader approach is presented by Zeng et al. [7] whose E2EDevBench evaluates both functional correctness and the proportion of requirements successfully implemented. Their findings highlight the limitations of relying on automated tests produced during development. Tests generated by the agents achieved a 93.14% pass rate, whilst additional tests adapted from the reference projects achieved only 59.66%. As illustrated in Figure 2, the difference between the test suites highlights the limitations of relying solely on agent-generated tests to evaluate functional correctness. This implies that agent-generated tests may fail to identify defects in their own implementations. Compared with SWE-bench, E2EDevBench provides a more comprehensive assessment of requirements fulfilment, although neither framework directly measures adherence to human oversight or development procedures.

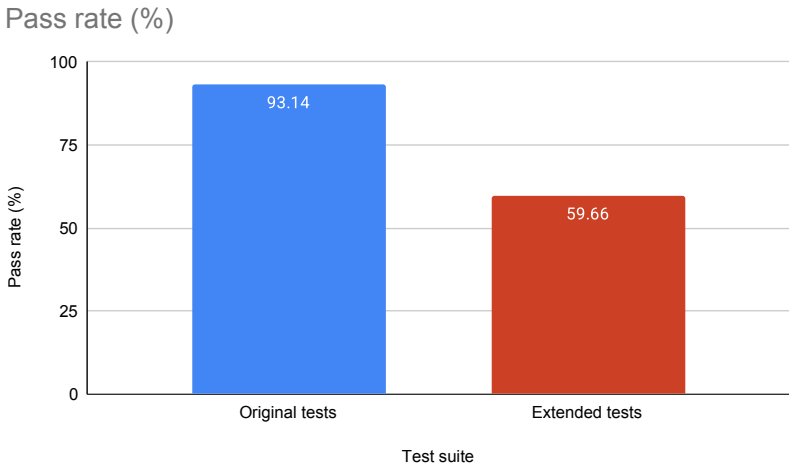


Figure 2: Pass rates for original agent-generated tests and extended tests in E2EDevBench, adapted from Zeng et al. [7].

However, evaluating requirements fulfilment introduces further challenges. Zeng et al. [7] use an LLM-based evaluator to determine whether individual requirements have been implemented. The authors compared these judgements against human assessments, finding agreement rates ranging from approximately 77% to 84%. Although this supports the usefulness of LLM-based evaluation, the validation involved only ten generated projects, meaning its reliability across other development contexts remains uncertain. The study also shows the importance of controlling differences in agent tools and environments when comparing development approaches, as these factors could influence results independently of the workflow being evaluated.

A comparison of development workflows was conducted by Mock and Russo [12], who evaluated different approaches to test-driven development, including human-only, collaborative, fully automated, and agentic workflows. Their findings imply that agentic approaches can achieve greater functional correctness and development speed, although this may come at the cost of increased code complexity and reduced test coverage. However, the human-centered and autonomous workflows were evaluated through different experimental settings, limiting any direct comparisons between them. This highlights the importance of evaluating multiple quality measures whilst maintaining consistent experimental conditions.

These findings suggest that evaluating an agent-assisted development workflow requires measures covering both the resulting software and the development process. Functional correctness and requirements coverage can be assessed through independent tests and acceptance criteria, whilst process adherence, human interventions, and guardrail effectiveness require additional evidence. Human effort and development time should also be considered, particularly when evaluating the benefits of increased oversight. By combining these measures within a controlled comparison, the proposed project aims to investigate whether a structured agent-assisted workflow can improve development outcomes whilst remaining consistent with established software engineering practices.

8 Research Gaps and Conclusions

Existing research shows that AI agents can undertake software development tasks using structured workflows, whilst different approaches have been proposed to manage their behaviour. MetaGPT [8] explores structured collaboration between agents, whereas HULA [5] incorporates human review and approval. More recently, Stozle and Strässle [10] identified layered supervision involving preventive restrictions, executable checks, and human oversight. Together, these studies demonstrate different approaches to managing agentic development

However, evidence regarding the effectiveness of combining these approaches remains limited. Stolze and Strässle's findings are based on qualitative interviews with five practitioners rather than controlled experimentation. Similarly, Mock and Russo [12] compared different AI-assisted development workflows, identifying trade-offs between functional correctness, code complexity, test quality, and efficiency. However, their research focuses on test-driven development, with human centred and autonomous approaches examined under different experimental conditions. Consequently, further investigation is needed into how combining established software engineering practices, human oversight, and technical guardrails affects development outcomes

This project aims to investigate this gap by developing a structured agent-assisted workflow informed by established software engineering practices. The workflow will be applied to the development of a volunteer accommodation booking system and evaluated against a less constrained approach. The evaluation will consider functional correctness, requirements fulfilment, process adherence, human intervention, and development efficiency. Given the scope of a single case study, the findings will provide exploratory evidence rather than conclusions applicable to all agentic software development.

9 Project Specification

9.1 Motivation, Aim and Objectives

As agentic AI becomes increasingly capable of performing software development tasks, there is a need to understand how established software engineering practices can be maintained. Although existing research explores different approaches to agent autonomy and supervision, evidence of their combined effectiveness remains limited. Investigating these approaches could help identify how agentic AI can be integrated into software development whilst maintaining software quality and development efficiency.

The aim of this project is to investigate how established software engineering practices, human oversight, and technical guardrails can be applied when agentic AI undertakes a substantial proportion of software implementation. This will be achieved through the development and empirical evaluation of a structured agent-assisted development workflow.

The project has six main objectives. Firstly, to review established software engineering practices and agentic AI research, identifying relevant approaches, limitations, and evaluation methods. Secondly, to define a structured workflow outlining the responsibilities of AI agents and human developers, incorporating guardrails, approval stages, testing, and code review. Thirdly, to establish the requirements and acceptance criteria for the Llechfan accommodation booking system and implement a demonstrable application using agentic AI.

Fourthly, to design and conduct an empirical evaluation, potentially comparing the structured workflow against a less constrained approach using comparable development tasks. Fifthly, to measure functional correctness, process adherence, guardrail effectiveness, human intervention and development efficiency. Finally, to analyse the findings against established software engineering principles, identifying the strengths, limitations, and trade-offs of the proposed approach.

9.2 Project Scope and Proposed System

The proposed practical application is an accommodation booking system for Llechfan, which provides volunteer accommodation consisting of three rooms, and twelve beds. Currently, bookings are coordinated through email, requiring administrators to manage reservations and room allocations manually. This approach presents potential difficulties in maintaining accurate booking records, managing availability, and preventing conflicting reservations. A centralised booking system could simplify these administrative processes and provide a more reliable method of managing accommodation bookings.

The proposed system will be a web-based application designed to centralise accommodation bookings and administration. Its anticipated functionality includes managing reservations, checking availability, allocating rooms or beds, and providing booking confirmations. A relational database is proposed to manage guest, accommodation, and reservation information, with consideration given to preventing conflicting allocations and maintaining data integrity. Security and access permissions will also need to be considered, particularly when handling guests' personal information. Before implementation, functional and non-functional requirements will be established and validated against stakeholder needs, alongside measurable acceptance criteria.

The Llechfan booking system provides a suitable practical case study for investigating agentic software development, as its implementation involves requirements engineering, system design, database management, and testing. These activities provide development tasks of varying complexity, through which the proposed workflow can be evaluated. By comparing agent performance under different development conditions, the project aims to investigate how structured SWE practices, human oversight, and technical guardrails affect development outcomes. The primary focus of the research is therefore the effectiveness of the agent-assisted workflow, rather than the booking system itself.

9.3 Development Methodology and Evaluation Criteria

The proposed methodology involves developing a structured agent-assisted workflow informed by the established software engineering practices and approaches reviewed from the literature previously. Requirements engineering and high level design decisions will remain human-led, whilst the AI agent will undertake specified implementation tasks, including code generation, testing, and debugging. Development will follow defined stages of task planning, implementation, automated testing, code review, and human approval before changes are integrated. Technical guardrails, such as restrictions on file access and tool permissions, will be introduced to limit unintended agent behaviours. Agent actions and human interventions will also be recorded to support the subsequent experimental evaluation.

To evaluate the proposed workflow, an experimental comparison will be conducted between structured and less constrained agentic development approaches. Both approaches will undertake the same implementation tasks derived from the Llechfan booking system, using predefined requirements and acceptance criteria. The structured approach will include approval stages, testing procedures, and technical guardrails, whilst the less constrained approach will allow greater agent autonomy. To support a fair comparison, factors such as the AI model, development environment, initial codebase, and task requirements will be kept consistent where possible. Pilot experiments will be conducted to assess the feasibility of the methodology, with repeated trials undertaken where resources permit.

The effectiveness of each development approach will be evaluated using predefined criteria covering functional correctness, process adherence, human intervention, guardrail effectiveness, and development efficiency. Functional correctness and requirements fulfilment will be assessed through automated tests and acceptance criteria established independently of the agent's implementation. Agent execution logs will be examined to identify deviations from prescribed procedures and attempted actions prevented by technical guardrails. The effectiveness of these guardrails will also be tested through predefined attempts to violate selected restrictions within an isolated development environment, recording whether the prohibited actions are successfully prevented. Human involvement will be recorded through the frequency and duration of the interventions, distinguishing routine approvals from corrective actions. Development efficiency will be assessed through task completion time and resource usage. These measurements will provide quantitative evidence for comparing the approaches and identifying potential trade-offs between reliability, oversight, and efficiency. The project will be considered successful if a demonstrable booking system is produced and the experimental evaluation provides sufficient comparable data to assess the effects of the structured workflow, regardless of whether it outperforms the less constrained approach.

Several limitations must be considered when interpreting the experimental findings. Agentic AI systems may produce different outcomes when undertaking identical tasks, potentially affecting the consistency of results. To mitigate this, experimental conditions and evaluation procedures will be standardised, with repeated trials conducted where feasible. Human involvement will also be documented to account for potential differences in reviewer decisions. Furthermore, the evaluation will focus on a single practical application, limiting the generalisability of the findings. As the structured workflow incorporates multiple forms of oversight and restriction, the experiment will primarily evaluate their combined effectiveness rather than the contribution of individual measures.

9.4 Deliverables, Timeline, Risks and Ethics

The expected project deliverables include a literature review and project specification, followed by a documented agent-assisted development workflow outlining the responsibilities of human developers and AI agents. A functional Llechfan accommodation booking system will be developed as the practical application, supported by requirements documentation and source code. An experimental methodology will be produced, alongside collected test results, agent execution logs, and records of human intervention. These findings will be analysed and presented in the final project report, accompanied by a project poster and practical demonstration.

The project will be completed across three academic terms, following a series of defined milestones. During term 1, the literature review and project specification will be finalised for submission on 17 November 2026. Term 2 will focus on establishing the system requirements and agent-assisted development workflow, with the core booking system targeted for completion for Week 6. Pilot experiments will be conducted in Week 7, followed by the main experimental evaluation in Week 9, and analysis of the findings in Week 10. The project poster will be completed by Week 11, with the final report, supporting code, and demonstration scheduled for Term 3, Week 1. These milestones establish dependencies between implementation, experimentation, and evaluation, allowing progress and potential delays to be monitored.

Several risks have been identified that could affect the project's implementation and evaluation. Incorrect AI-generated code, unintended agent behaviour, and technical complexity present significant risks, which will be mitigated through automated testing, human review, restricted agent permissions, and clearly defined requirements. Scope creep and scheduling delays will be managed by prioritising essential functionality and monitoring progress against project milestones. Experimental validity will be addressed through consistent evaluation procedures and pilot testing. Ethical considerations primarily concern the handling of personal information within the booking system and the responsible use of agentic AI. Synthetic booking data will be used during experimentation where possible to minimise privacy risks. Formal ethical approval is not currently required, although this will be reconsidered if the research methodology changes.

References

[1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024. [Online]. Available: https://doi.org/10.1145/3695988

[2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

[3] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0.* IEEE Computer Society, 2024. [Online]. Available: https://www.computer.org/education/bodies-of-knowledge/software-engineering/v4

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/edac78c3e300629acfe6cbe9ca88fb84-Paper-Conference.pdf

[5] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-in-the-Loop Software Development Agents," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 342–352. [Online]. Available: https://doi.org/10.1109/ICSE-SEIP66354.2025.00036

[6] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026, pp. 2938–2950. [Online]. Available: https://doi.org/10.1145/3744916.3764546

[7] Z. Zeng, Y. Li, R. Xie, W. Ye, and S. Zhang, "Benchmarking and Studying the LLM-based Agent System in End-to-End Software Development," 2025. [Online]. Available: https://arxiv.org/abs/2511.04064

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html

[9] U. Cihan, V. Haratian, A. İçöz, M. K. Gül, Ö. Devran, E. F. Bayendur, B. M. Uçar, and E. Tüzün, "Automated Code Review in Practice," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025, pp. 425–436. [Online]. Available: https://doi.org/10.1109/ICSE-SEIP66354.2025.00043

[10] M. Stolze and M. Strässle, "When Review Alone No Longer Scales: Layered Supervision in AI-Assisted Software Engineering," in *20th International Symposium on Empirical Software Engineering and Measurement (ESEM 2026)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 394. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 90:1–90:12. [Online]. Available: https://doi.org/10.4230/LIPIcs.ESEM.2026.90

[11] L. Pasquale, A. Ragone, E. Piemontese, and A. A. Darban, "Exploring the Use of LLMs for Requirements Specification in an IT Consulting Company," in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, 2025, pp. 389–399. [Online]. Available: https://doi.org/10.1109/RE63999.2025.00045

[12] M. Mock and B. Russo, "Vibe Coding: An Experiment with Test-Driven Development," *Empirical Software Engineering*, vol. 32, 2027, article 40; published online 9 October 2026. [Online]. Available: https://doi.org/10.1007/s10664-026-10950-z